

Token Budget *Optimization* in LLM Apps

How real GenAI systems manage context limits — and why it matters for cost, speed, and accuracy.

Cost

Wasted tokens =
wasted money

Speed

Longer context =
slower response

Quality

Bloated context
= less accurate

Chunking

Reranking

Compression

Caching

Prompt Opt.

Monitoring

Understanding Context Limits

Every LLM has a maximum context window. Everything must fit — system prompt, user input, retrieved docs, history, and output.

MODEL LIMITS

GPT-4 → 128k tokens

Claude → 200k tokens

Small models → 4k–32k

WHAT FILLS THE WINDOW

System prompt

+ User input

+ Retrieved docs

+ Conversation history

+ Model output

→ Design your system to allocate token budgets per component — not send everything at once.

One Kit to Clear Your AI Interview

After seeing how AI interviews are evolving, I created an **AI Interview Kit**. It covers the areas companies actually test:

AI fundamentals

AI agents

Transformers (in depth)

Memory systems

LLM architectures

Databases

Prompt engineering

Backend for AI systems

Fine-tuning

Deployment strategies

RAG systems

System design for GenAI

Everything structured to help you prepare.
Check the caption for the AI Interview Kit!

Token Budget Allocation

Before sending a request, split the context budget. Example for a **32k context model**:



Total: 32k tokens allocated

SYSTEM PROMPT

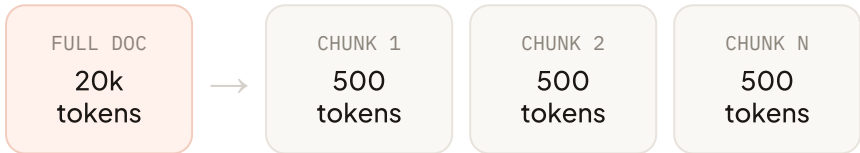
Keep it tight. 500–1k tokens is plenty for most use cases.

RETRIEVED DOCS

Gets the largest slice — prioritize what's relevant, not all docs.

Chunking Documents

Large documents must be split into smaller chunks before retrieval. Only the most relevant chunks get sent to the LLM — not the whole document.



```
chunks = split_document(  
    text,  
    chunk_size=500,  
    overlap=50 # slight overlap preserves context  
)
```

Chunk Ranking

Not all retrieved chunks are equally useful. Reranking filters the top-20 results down to only the 5 most relevant before sending to the LLM.

QUERY User question



RETRIEVE Vector search → top 20 chunks



RERANK Reranker → top 5 chunks sent to LLM

RERANKER TOOLS

Cohere Rerank · Cross-encoder rerankers · LLM-based reranking

Dynamic Retrieval

Instead of a fixed number of chunks, adjust retrieval size based on query complexity. Simple questions need fewer tokens; hard ones need more.

```
if query_complexity == "high":
    retrieve_k = 8    # more context needed
elif query_complexity == "medium":
    retrieve_k = 5
else:
    retrieve_k = 3    # simple query, fewer tokens

docs = retrieve(query, k=retrieve_k)
```

SIMPLE QUERY

"What is RAG?" → retrieve
3 chunks. No need for 20.

COMPLEX QUERY

"Compare all retrieval
strategies" → retrieve 8+
chunks.

Context Compression

Large retrieved chunks can be summarized before they reach the LLM. Fit more useful information into the same context window.

BEFORE COMPRESSION

Original chunk
→ 500 tokens

AFTER COMPRESSION

Compressed summary
→ 120 tokens

```
# Summarize each chunk before injecting
compressed = []
for chunk in retrieved_chunks:
    summary = llm.summarize(chunk)
    compressed.append(summary)

send_to_llm(compressed) # ~4x fewer tokens
```


Summarizing Conversation History

Chat systems accumulate long histories fast. Instead of sending all prior messages, compress them into a rolling summary.

FULL HISTORY · 2000 tokens



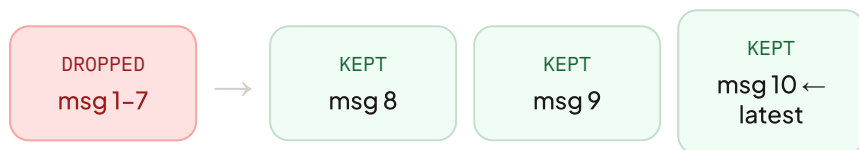
SUMMARY · 300 tokens

```
history_summary = summarize_chat(history)

# Keep: recent messages + compressed past
context = {
    "summary": history_summary,
    "recent": history[-3:]
}
```

Sliding Window History

When older messages aren't needed, just keep the last N turns. Simple, fast, and surprisingly effective for most chat use cases.



```
# Keep only last 5 messages
history = history[-5:]

# Or by token count
while count_tokens(history) > 2000:
    history.pop(0)  # drop oldest
```

Caching Responses

Many user queries repeat. Serving from cache avoids the LLM call entirely — saving tokens, latency, and compute cost.

```
import hashlib

def cached_llm(query):
    key = hashlib.md5(query.encode()).hexdigest()
    if key in cache:
        return cache[key]  # instant, zero tokens
    result = llm(query)
    cache[key] = result
    return result
```

TOKENS

Zero prompt
tokens on
cache hit

LATENCY

Milliseconds vs
seconds

COST

Cached calls
cost nothing

Prompt Optimization

Long, verbose prompts waste tokens on every single request. Short, precise prompts reduce cost and latency without hurting quality.

✗ VERBOSE (BAD)

You are a helpful AI assistant carefully designed to analyze and summarize text content...

✓ CONCISE (GOOD)

Summarize the following text.

- Remove filler phrases ("please", "as an AI", "carefully")
- Avoid repeating instructions that are already implied
- Use structured formats (JSON schema) instead of long explanations

Output Length Control

Model responses can also consume a large share of tokens. Set `max_tokens` based on the task — don't let the model ramble.

```
response = llm.generate(  
    prompt,  
    max_tokens=300,          # hard cap  
    stop=["\n\n", "---"]    # stop sequences  
)
```

SUMMARIES

200 tokens
max

ANSWERS

500 tokens
max

REPORTS

1000 tokens
max

Multi-Stage Generation

Instead of sending a massive context to the LLM in one shot, split the process into stages. Each stage passes only what's needed forward.

STEP 1 Retrieve relevant documents



STEP 2 Summarize docs (compress context)



STEP 3 Generate final answer from summary

```
docs      = retrieve(query)
summary   = summarize(docs)    # small, focused
answer    = generate(summary)  # no raw docs
```

Token Monitoring

Track token usage across every request. Monitoring helps you spot expensive prompts, detect regressions, and justify optimization work.

```
def log_tokens(response, query_id):
    usage = response.usage
    metrics.record({
        "query_id":    query_id,
        "prompt_tok":  usage.prompt_tokens,
        "output_tok":  usage.completion_tokens,
        "total_tok":   usage.total_tokens,
        "cost_usd":    calc_cost(usage)
    })
```

- Alert on token spikes (prompt grew unexpectedly)
- Track cost per query type and endpoint
- A/B test different retrieval configs by token efficiency

Why Token Optimization Matters

In production AI systems, **tokens are the most expensive resource**. Every technique here directly reduces cost, latency, or improves accuracy.

01

Chunking

Split docs, send only relevant pieces

02

Reranking

Top-K filtering before LLM call

03

Dynamic Retrieval

Adjust K by query complexity

04

Compression

Summarize chunks before injection

05

History Mgmt

Summarize or window old turns

06

Caching

Zero tokens on repeated queries

07

Prompt Opt.

Short precise prompts always win

08

Output Limits

Cap max_tokens per task type

09

Multi-Stage

Pipeline: retrieve → compress → generate